

操作系统

2020最新南开大学期末最全知识点总结，吐血整理，包上90分！里面有短期调度器，进程行为，Cpu密集型&IO密集型，抢占式，非抢占式，先来先服务（FIFO），抢占式的最短剩余时间优先，轮转调度，优先级调度，多级队列调度，内核级线程，用户级线程，死锁，死锁检测&死锁恢复，银行家算法，基址寄存器&界限寄存器，虚拟内存，MUU（内存管理单元），硬件TLB，页面置换算法，软件实现LRU，工作集时钟页面置换算法，文件系统，必考：程序控制 i / o，电梯算法！！！考

填空

一堆简答

为什么分两种状态

进程几种状态

系统调用

Linux内核（上机）

文件包括哪几部分 文件控制块

系统调用

名词解释

并发共享虚拟异步

库函数

进程线程 差异 比较

调度算法

IPC

死锁

进程控制块PCB

用户级线程和**内核级**线程 怎么实现，区别

多进程到多线程 库函数为什么需要改，有什么区别

哲学家 读写者 **理发师**

死锁必要条件 解决方案 银行家算法

基本存在管理的特点： 连续的一次性载入

虚拟存储：局部性

balady

文件系统

fat表原理 i节点原理计算 二选一

空闲块管理：成组链接法

磁盘调度

i/o控制方式

假脱机技术 原理 例子

未来发展方向

多核 虚拟化

中断和陷阱的区别：

中断：与当前执行程序无关

陷阱traps：当前执行的程序异常中断

计算机系统概述

操作系统：

- 最基本的系统软件
- 系统资源管理者（控制）
 - 文件管理、内存管理、处理机（CPU）管理、硬件设备管理（摄像头、打印机）
- 用户和计算机硬件的接口
- 四个特征
 - 并发：宏观同时，微观交替
 - 共享：资源共享（分时共享）
 - 虚拟：
 - 异步：并发执行，

运行机制和体系结构：

- 两种指令
 - 特权指令：如内存清零指令（禁止用户程序使用）
 - 非特权指令：如运算指令（用户程序使用的指令）
- CPU处理器的两种状态（状态寄存器PSW的标识位）
 - 用户态：此时CPU只能执行非特权指令
 - 核心态：特权指令、非特权指令都课执行
- 两种程序
 - 应用程序（用户程序）：非特权指令 用户态
 - 内核程序：内核程序是系统的的管理者 特权指令、非特权指令 核心态

中断和异常：

意外——主机干预——暂停原程序——中断服务程序——返回原程序

中断发生后，CPU立即进入核心态

要想实现“用户态——核心态”，必须经过中断，执行特权指令，将PSW的标志位设置为“用户态”

- 两种中断类型
 - 内中断：

系统调用：

操作系统提供给应用程序（编程人员）调用请求使用操作系统内核服务的接口。

- 系统调用发生在用户态，对系统调用的处理发生在核心态。
- 执行陷入指令（自陷指令或访管指令）会处理内中断，使处理器（CPU）从用户态进入核心态。

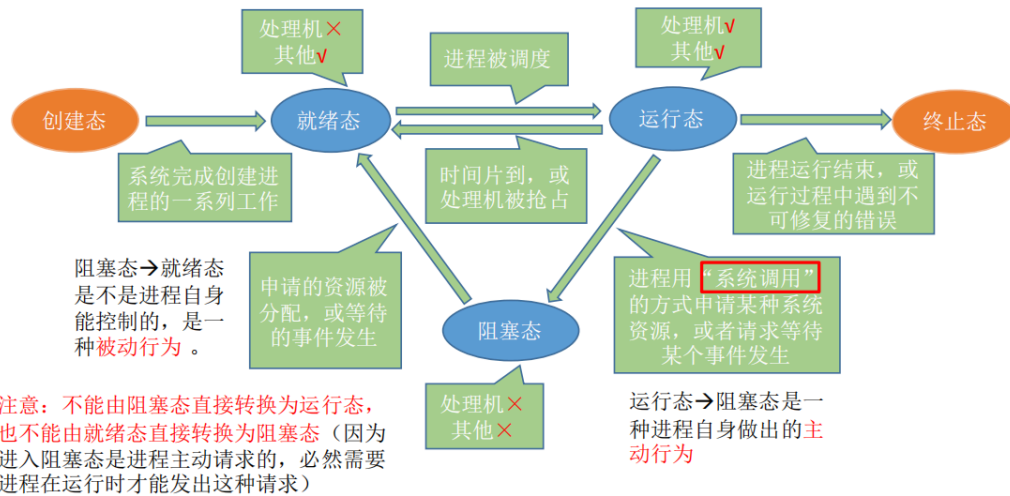
进程管理

进程：运行着的程序。

PCB是进程存在的唯一标识。

进程控制就是要实现进程状态转换。原语——核心态

进程通信就是进程之间的信息交换



线程概念和多线程模型

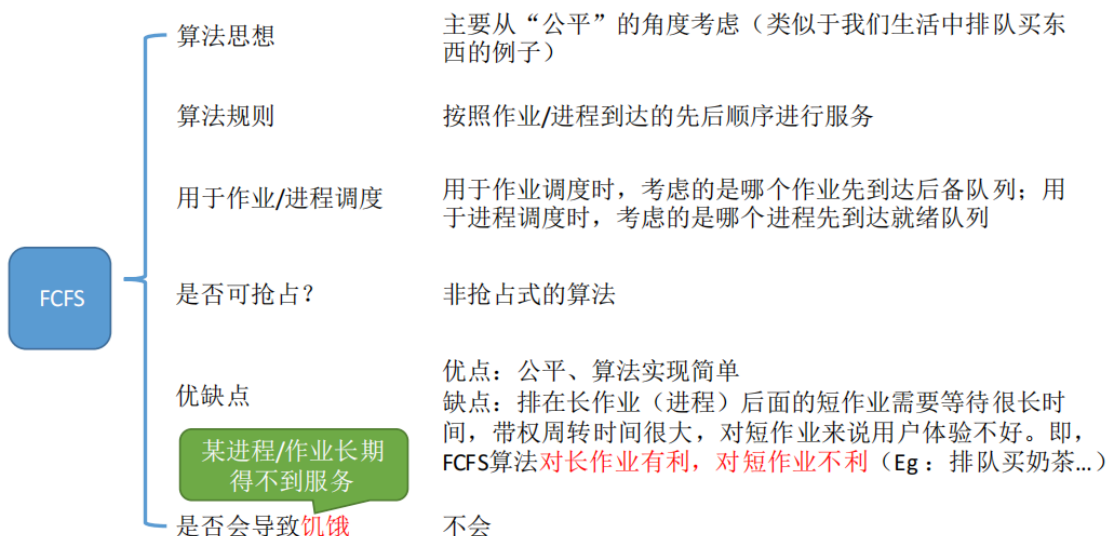
- 概念
 - 线程可以理解为轻量级的进程
 - 线程是一个基本的CPU执行单元，也是程序执行流的最小单位
 - 引入线程后，进程作为除CPU之外的系统资源的分配单元
- 用户级线程
 - 用户态 无需操作系统干预
- 内核级线程
 - 操作系统内核完成
- 多对一模型
 - 多个用户级线程映射到一个内核级线程（每个用户级进程只对应一个内核级线程）
 - 优点:用户级线程的切换在用户空间即可完成，不需要切换到核心态，**线程管理的系统开销小，效率高**
 - 缺点:当一个用户级线程被阻塞后，整个进程都会被阻塞，并发度不高。**多个线程不可在多核处理机上并行运行**
- 一对一模型
 - 一个用户级线程映射到一个内核级线程
 - 优点:当一个线程被阻塞后，别的线程还可以继续执行，并发能力强。**多线程可在多核处理机上并行执行。**
 - 缺点:一个用户进程会占用多个内核级线程，线程切换由操作系统内核完成，需要切换到核心态，**因此线程管理的成本高，开销大。**

- 多对多模型
 - 多对多模型:n用户级线程映射到m个内核级线程($n \geq m$)。每个用户进程对应m个内核级线程。
 - 克服了多对一模型并发度不高的缺点, 又克服了一对一模型中一个用户进程占用太多内核级线程, 开销太大的缺点。

处理机调度的概念和层次:

调度算法

• 先来先服务 (FCFS, First Come First Serve)



• 短作业优先 (SJF, Shortest Job First)

短作业优先 (SJF, Shortest Job First)



• 时间片轮转调度 (RR, Round-Robin)

优先级调度	算法思想	公平地、轮流地为各个进程服务，让每个进程在一定时间间隔内都可以得到响应
	算法规则	按照各进程到达就绪队列的顺序，轮流让各个进程执行一个时间片（如 100ms）。若进程未在一个时间片内执行完，则剥夺处理机，将进程重新放到就绪队列队尾重新排队。
	用于作业/进程调度	用于进程调度（只有作业放入内存建立了相应的进程后，才能被分配处理机时间片）
	是否可抢占？	若进程未能在时间片内运行完，将被强行剥夺处理机使用权，因此时间片轮转调度算法属于 抢占式 的算法。由时钟装置发出 时钟中断 来通知CPU时间片已到
	优缺点	优点：公平；响应快，适用于分时操作系统； 缺点：由于高频率的进程切换，因此有一定开销；不区分任务的紧急程度。
	是否会导致饥饿	不会
	补充	时间片太大或太小分别有什么影响？

• 优先级调度算法

优先级调度	算法思想	随着计算机的发展，特别是实时操作系统的出现，越来越多的应用场景需要根据任务的紧急程度来决定处理顺序
	算法规则	调度时选择优先级最高的作业/进程
	用于作业/进程调度	既可用于作业调度，也可用于进程调度。甚至，还会用于在之后会学习的I/O调度中
	是否可抢占？	抢占式、非抢占式都有。做题时的区别在于：非抢占式只需在进程主动放弃处理机时进行调度即可，而抢占式还需在就绪队列变化时，检查是否会发生抢占。
	优缺点	优点：用优先级区分紧急程度、重要程度，适用于实时操作系统。可灵活地调整对各种作业/进程的偏好程度。 缺点：若源源不断地有高优先级进程到来，则可能导致饥饿
	是否会导致饥饿	会

• 多级反馈队列调度算法

多级反馈队列	算法思想	对其他调度算法的折中权衡
	算法规则	1. 设置多级就绪队列，各级队列优先级从高到低，时间片从小到大 2. 新进程到达时先进入第1级队列，按FCFS原则排队等待被分配时间片，若用完时间片进程还未结束，则进程进入下一级队列队尾。如果此时已经是在最下级的队列，则重新放回该队列队尾 3. 只有第 k 级队列为空时，才会为 $k+1$ 级队头的进程分配时间片
	用于作业/进程调度	用于进程调度
	是否可抢占？	抢占式 的算法。在 k 级队列的进程运行过程中，若更上级的队列（ $1 \sim k-1$ 级）中进入了一个新进程，则由于新进程处于优先级更高的队列中，因此新进程会抢占处理机，原来运行的进程放回 k 级队列队尾。
	优缺点	对各类型进程相对公平（FCFS的优点）；每个新到达的进程都可以很快就得到响应（RR的优点）；短进程只用较少的时间就可完成（SPF的优点）；不必实现估计进程的运行时间（避免用户作假）；可灵活地调整对各类进程的偏好程度，比如CPU密集型进程、I/O密集型进程（拓展：可以将因I/O而阻塞的进程重新放回原队列，这样I/O型进程就可以保持较高优先级）
	是否会导致饥饿	会

进程同步与进程互斥

- 同步
 - 按顺序执行
 - 异步：顺序不确定
- 互斥
 - 临界资源互斥访问
 - 空闲让进（临界区空闲，请求则允许访问）
 - 忙则等待（临界区忙，阻塞等待）
 - 有限等待（阻塞等待时间的有限，避免饥饿）（受惠的是进程自己）
 - 让权等待（当进程不能进入临界区时，应立即释放处理机，避免忙等待）（受惠的是其他进程）

进程互斥的软件实现方法

单标志法

算法思想：每个进程在访问完临界区后会把使用临界区的权限转交给另一个进程。也就是说**每个进程进入临界区的权限只能被另一个进程赋予**。

```
int turn = 0; //turn 表示当前允许进入临界区的进程号
```

P0 进程：

```
while (turn != 0);    ①
critical section;    ②
turn = 1;             ③
remainder section;   ④
```

P1进程：

```
while (turn != 1);    ⑤ //进入区
critical section;    ⑥ //临界区
turn = 0;             ⑦ //退出区
remainder section;   ⑧ //剩余区
```

单标志法可以实现“同一时刻最多只允许一个进程访问临界区”，但违背“空闲让进”原则。

双标志先检查法

信号量机制

- 信号量机制实现进程互斥
 - 划定临界区（如：对临界资源打印机的访问就应放在临界区）
 - 设置互斥信号量mutex，初值为1
 - 在临界区之前执行P(mutex) wait
 - 在临界区之后执行V(mutex) signal
- 信号量机制实现进程同步

进程同步：要让各并发进程按要求有序的进行。

- 分析什么地方需要实现“同步关系”，即必须保证“一前一后”执行的两个操作（或两句代码）
- 设置同步信号量s,初始为0
- 在“前操作”之后执行v(S)
- 在“后操作”之前执行P(S)

生产者——消费者问题

吸烟者问题

读者——写者问题

哲学家吃饭问题

死锁 deadlock

- 含义
 - 在并发环境下，各进程因竞争资源而造成的一种互相等待对方手里的资源，导致各进程都阻塞，都无法向前推进的现象，就是“死锁”。
- 死锁，饥饿，死循环的区别
 - **死锁**：互相等待对方手里的资源，导致阻塞，无法正常向前推进
 - **饥饿**：长期得不到想要的资源，无法正常向前推进。比如：SPF算法中长进程可能一直得不到处理，发生“饥饿”
 - **死循环**：程序逻辑bug
- 死锁的必要条件（同时满足4个条件）
 - **互斥**：必须互斥使用的资源（哲学家的筷子、打印机设备）
 - **不剥夺**：互斥资源没有使用完，只能主动释放，其他进程不能夺走
 - **请求和保持**：已经拥有至少一个互斥资源，但又有新的互斥资源请求，该资源又被其他进程占有，此时请求进程被阻塞，但又对自己已经拥有的资源保持占有
 - **循环等待**：资源的循环等待链，每一个进程拥有的资源被另一个进程请求

注意：死锁一定有循环等待，循环等待未必死锁（同类资源数大于1）

- 什么时候会发生死锁
 - 对不可剥夺的互斥资源的竞争
 - 进程推进顺序非法

- 信号量使用不当
- 死锁的处理策略
 - 预防死锁：破坏四个条件之一
 - 互斥：假脱机（虚拟）
 - 不剥夺：抢占式
 - 请求和保持：一次性请求分配所需的所有资源
 - 循环等待：资源编号，按序访问
 - 避免死锁：**安全序列**（安全状态一定不会死锁）**A R C E 银行家算法 资源分配图**
 - 检测和恢复：发生死锁后采取某种措施解除死锁
 - 资源剥夺法
 - 终止进程法
 - 回退进程法

内存管理

内存是用于存放数据的硬件。程序执行前需要先放到内存中才能被CPU处理。

- 内存保护的两种方法
 - 在CPU中设置一对上、下限寄存器，存放进程的上、下限地址。进程的指令要访问某个地址时，CPU检查是否越界。
 - 采用重定位寄存器（又称基址寄存器）和界地址寄存器（又称限长寄存器）进行越界检查。重定位寄存器中存放的是进程的起始物理地址。界地址寄存器中存放的是进程的最大逻辑地址。

连续分配管理方式

连续分配：为用户进程分配的必须是一个连续的内存空间。

分区

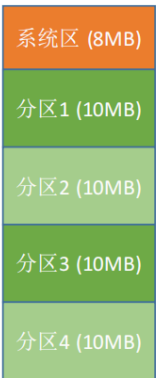
• 固定分区分配

20世纪60年代出现了支持多道程序的系统，为了能在内存中装入多道程序，且这些程序之间又不会相互干扰，于是将整个用户空间划分为若干个固定大小的分区，在每个分区中只装入一道作业，这样就形成了最早的、最简单的一种可运行多道程序的内存管理方式。



分区大小相等：缺乏灵活性，但是很适用于用一台计算机控制多个相同对象的场合（比如：钢铁厂有n个相同的炼钢炉，就可把内存分为n个大小相等的区域存放n个炼钢炉控制程序）

分区大小不等：增加了灵活性，可以满足不同大小的进程需求。根据常在系统中运行的作业大小情况进行划分（比如：划分多个小分区、适量中等分区、少量大分区）



内存（分区大小相等）



内存（分区大小不等）

操作系统需要建立一个数据结构——分区说明表，来实现各个分区的分配与回收。每个表项对应一个分区，通常按分区大小排列。每个表项包括对应分区的大小、起始地址、状态（是否已分配）。

分区号	大小（MB）	起始地址（M）	状态
1	2	8	未分配
2	2	10	未分配
3	4	12	已分配
.....			

用数据结构的数组（或链表）即可表示这个表

当某用户程序要装入内存时，由操作系统内核程序根据用户程序大小检索该表，从中找到一个能满足大小的、未分配的分区，将之分配给该程序，然后修改状态为“已分配”。

优点：实现简单，无外部碎片。

缺点：a. 当用户程序太大时，可能所有的分区都不能满足需求，此时不得不采用覆盖技术来解决，但这又会降低性能；b. 会产生内部碎片，内存利用率低。



内存（分区大小不等）

• 动态分区分配（可变分区分配）

- 根据进程的大小动态地建立分区，并使分区的大小正好适合进程的需要。因此系统分区的大小和数目是可变的。

• 动态分区分配算法

- 首次适应算法 First Fit
 - 每次都从低地址开始查找，找到第一个能满足大小的空闲分区。
- 临近适应算法 Next Fit
 - 首次适应算法每次都从链头开始查找的。这可能会导致低地址部分出现很多小的空闲分区，而每次分配查找时，都要经过这些分区，因此也增加了查找的开销。**如果每次都从上次查找结束的位置开始检索，就能解决上述问题。**
- 最差（大）适应算法 Worst/Largest Fit
 - 为了解决最佳适应算法的问题——即留下太多难以利用的小碎片，可以在**每次分配时优先使用最大的连续空闲区**，这样分配后剩余的空闲区就不会太小，更方便使用。
- 最佳适应算法 Best Fit
 - 由于动态分区分配是一种连续分配方式，为各进程分配的空间必须是连续的一整片区域。因此为了保证当“大进程”到来时能有连续的大片空间，可以尽可能多地留下大片的空闲区，即，**优先使用更小的空闲区**。
 - 碎片

- 内部碎片，分配给某进程的内存区域中，如果有些部分没有用上。
外部碎片，是指内存中的某些空闲分区由于太小而难以利用。
- 动态分区分配没有内部碎片，但是有外部碎片。
- 如果内存中空闲空间的总和本来可以满足某进程的要求，但由于进程需要的是
一整块连续的内存空间，因此这些
进程“碎片”不能满足进程的需求。可以通过紧凑 ((拼凑, Compaction)技术来
解决外部碎片。
 - 快速适应算法
 - 买鞋

算法	算法思想	分区排列顺序	优点	缺点
首次适应	从头到尾找适合的分区	空闲分区以地址递增次序排列	综合看性能最好。 算法开销小 ，回收分区后一般不需要对空闲分区队列重新排序	
最佳适应	优先使用更小的分区，以保留更多大分区	空闲分区以容量递增次序排列	会有更多的大分区被保留下来，更能满足大进程需求	会产生很多太小的、难以利用的碎片； 算法开销大 ，回收分区后可能需要对空闲分区队列重新排序
最坏适应	优先使用更大的分区，以防止产生太小的不可用的碎片	空闲分区以容量递减次序排列	可以减少难以利用的小碎片	大分区容易被用完，不利于大进程； 算法开销大 （原因同上）
邻近适应	由首次适应演变而来，每次从上次查找结束位置开始查找	空闲分区以地址递增次序排列（可排列成循环链表）	不用每次都从低地址的小分区开始检索。 算法开销小 （原因同首次适应算法）	会使高地址的大分区也被用完

分页/分段存储

基本分页存储管理的思想——把内存分为一个个相等的小分区,再按照分区大小把进程拆分成一个个小部分。

各个页面不必连续存放，也不必按先后顺序来，可以放到不相邻的各个页框中。

具有快表的地址变换机构

1.局部性原理

时间局部性:如果执行了程序中的某条指令，那么不久后这条指令很有可能再次执行;如果某个数据被访问过，不久之后该数据很可能再次被访问。(因为程序中存在大量的循环)

空间局部性:一旦程序访问了某个存储单元，在不久之后，其附近的存储单元也很有可能被访问。(因为很多数据在内存中都是连续存放的)

上小节介绍的基本地址变换机构中，每次要访问一个逻辑地址，都需要查询内存中的页表。由于局部性原理，可能连续很多次查到的都是同一个页表项。既然如此，能否利用这个特性减少访问页表的次数呢?

2.快表

快表，又称联想寄存器 (TLB)，是一种访问速度比内存快很多的高速缓冲存储器，用来存放当前访问的若干页表项，以加速地址变换的过程。与此对应，内存中的页表常称为慢表。

多级页表：

- 第一次访存:访问内存中的页目录表
- 第二次访存:访问内存中的二级页表
- 第三次访存:访问目标内存单元

N级页表访问一个逻辑地址需要N+1次访问内存。

4.分段，分页对比

页是信息的物理单位。分页的主要目的是为了实现在离散分配，提高内存利用率。分页仅仅是系统管理上的需要，完全是系统行为，对用户是不可见的。

段是信息的逻辑单位。分段的主要目的是更好地满足用户需求。一个段通常包含着一组属于一个逻辑模块的信息。分段对用户是可见的，用户编程时需要显式地给出段名。

页的大小固定且由系统决定。段的长度却不固定，决定于用户编写的程序。

分页的用户进程地址空间是一维的，程序员只需给出一个记忆符即可表示一个地址。

分段的用户进程地址空间是二维的，程序员在标识一个地址时，既要给出段名，也要给出段内地址。

分段比分页更容易实现信息的共享和保护。

虚拟内存

请求分页管理

页面置换算法

LRU

FIFO

Belady异常——当为进程分配的物理块数增大时，缺页次数不减反增的异常现象。

只有FIFO算法会产生Belady异常。另外，FIFO算法虽然实现简单，但是该算法与进程实际运行时的规律不适应，因为先进入的页面也有可能最经常被访问。因此，**算法性能差。**

抖动

- 原因
 - 多进程 全局页面置换
 - 操作系统监控CPU利用率

刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出外存，这种频繁的页面调度行为称为抖动，或颠簸。产生抖动的主要原因是进程频繁访问的页面数目高于可用的物理块数（分配给进程的物理块不够）。

为进程分配的物理块太少，会使进程发生抖动现象。为进程分配的物理块太多，又会降低系统整体的并发度，降低某些资源的利用率

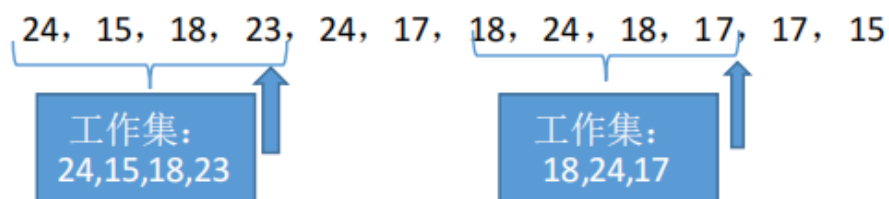
工作集时钟页面置换算法：

驻留集：指请求分页存储管理中给进程分配的内存块的集合。

工作集：指在某段时间间隔里，进程实际访问页面的集合。（LRU）

一般来说，驻留集大小不能小于工作集大小，否则进程运行过程中将频繁缺页。

操作系统会根据“窗口尺寸”来算出工作集。例：
某进程的页面访问序列如下，窗口尺寸为 4，各时刻的工作集为？



局部置换：发生缺页时只能选进程自己的物理块进行置换（进程内部置换）。

全局置换：可以将操作系统保留的空闲物理块分配给缺页进程，也可以将别的进程持有的物理块置换到外存，再分配给缺页进程。

文件系统

文件——信息的逻辑单元 字节流

混合索引 (I node)

成组连接法 原理

I/O管理

中断和陷阱的区别：

中断：与当前执行程序无关

陷阱traps：当前执行的程序异常中断

I/O控制方式

- **程序直接控制**
 - CPU发出I/O指令后需要不断轮询（忙等待）
- **中断驱动**
 - CPU发出I/O命令后可以做其他事，I/O完成后向CPU发出中断信号
- **DMA直接内存访问**
 - CPU发出I/O命令后可以做其他事，I/O完成后向CPU发出中断信号
 - 每次访存能够连续读取多个字（块）（降低CPU干预频率）
 - 一个连续块
- **通道控制**
 - 通道控制方式是了解决DMA方式连续存储的问题
 - 多个连续块

buffer

磁盘

柱面号，盘面号，扇区号

FCFS

SSF

Elevater(SCAN)

	完成一次读/写的过程	CPU干 预频率	每次I/O的数 据传输单位	数据流向	优缺点
程序直接控 制方式	CPU发出I/O命令后需要不 断轮询	极高	字	设备→CPU→内存 内存→CPU→设备	每一个阶段的 优点都是解决 了上一阶段的 最大缺点。 总体来说，整 个发展过程就 是要尽量减少 CPU对I/O过程 的干预，把CPU 从繁杂的I/O控 制事务中解脱 出来，以便更 多地去完成数 据处理任务。
中断驱动方 式	CPU发出I/O命令后可以 做其他事，本次I/O完成后 设备控制器发出中断信号	高	字	设备→CPU→内存 内存→CPU→设备	
DMA方式	CPU发出I/O命令后可以 做其他事，本次I/O完成后 DMA控制器发出中断信号	中	块	设备→内存 内存→设备	
通道控制方 式	CPU发出I/O命令后可以 做其他事。通道会执行通道 程序以完成I/O，完成后通 道向CPU发出中断信号	低	一组块	设备→内存 内存→设备	

难点理解：
通道=弱鸡版CPU
通道程序=任务清单